

Natural Language Processing

CSCI 5832—Lecture 9

Jim Martin



Today

- Word meanings and how we might capture them in representations that are useful computationally
 - Chapter 5: Embeddings/Vector Semantics
 - Dimensions of word meaning
 - Word embeddings

HW Submission

Releasing the test set today. Call you models/classifier as is based on what you submitted yesterday. Write code to read the test file and produce an output file to submit.

- One snippet per line
 - Item-id, \tab, text to classify
- You should output
 - Item-id \tab Author {Doyle or Tolkien}

Example

ITEM-10 In five miles we saw no rift or break. And then suddenly we perceived something which filled us with new hope. In a hollow of the rock, protected from rain, there was drawn a rough arrow in chalk, pointing still to the westwards.

ITEM-10 Doyle

Matrix Count Transformation with Mutual Information

Pointwise Mutual Information is the ratio of a joint probability of two events to their probability if we considered them as independent events.

$$I(x, y) = \log \left(\frac{P(x, y)}{P(x)P(y)} \right)$$

- At its core, mutual information tells us if 2 events happening together tells us anything interesting.

Pointwise Mutual Information

Let's adapt mutual information to the term-document matrix vector space setting.

$$\text{PMI}(w, d) = \log \left(\frac{P(w, d)}{P(w)P(d)} \right)$$

Pointwise Mutual Information

$$P(w, d) = \frac{\text{Count}(w, d)}{N}$$
$$\text{PMI}(w, d) = \log \left(\frac{P(w, d)}{P(w)P(d)} \right)$$
$$P(w) = \frac{\text{Count}(w)}{N}$$
$$P(d) = \frac{\text{Size}(d)}{N}$$

The diagram illustrates the components of the Pointwise Mutual Information (PMI) formula. Three equations are shown: the joint probability $P(w, d)$ at the top, the marginal probability $P(w)$ at the bottom left, and the marginal probability $P(d)$ at the bottom right. Blue arrows point from each of these three equations towards the central PMI formula, indicating that each is a component used in its calculation.

Pointwise Mutual Information

$$\text{PMI}(w, d) = \log \left(\frac{P(w, d)}{P(w)P(d)} \right)$$

How many instances of a word did we observe in some document?

$$= \log \left(\frac{\frac{\text{Count}(w, d)}{N}}{\frac{\text{Count}(w)}{N} \times \frac{\text{Size}(d)}{N}} \right)$$

How many instances of that word would we have expected in a document of size d by chance?

$$= \log \left(\frac{\text{Count}(w, d)}{\frac{\text{Count}(w)}{N} \times \text{Size}(d)} \right)$$

Term-Document Matrix

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
soldier	1	2	5	25
good	93	76	62	77
wit	21	15	2	3
fool	32	55	2	4

$$N = \sum_{i=1}^m \sum_{j=1}^n X_{i,j}$$

$$\text{Count}(w_i) = \sum_{j=1}^N X_{i,j}$$

$$\text{Size}(d_j) = \sum_{i=1}^M X_{i,j}$$

N = 496
Count(fool) = 93
Size(12th) = 148

Term-Document Matrix

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
soldier	1	2	5	25
good	93	76	62	77
wit	21	15	2	3
fool	32	55	2	4

$$\text{PMI}(w, d) = \log \left(\frac{\text{Count}(w, d)}{\frac{\text{Count}(w)}{N} \times \text{Size}(d)} \right) = \log_2 \left(\frac{55}{27.75} \right) = 0.987$$

PMI Entries

	asyou	twelfth	caesar	henryV
battle	-2.647574	-inf	1.083832	1.331581
soldier	-3.299651	-2.299651	-0.053672	1.622921
good	0.017115	-0.274116	0.356204	0.023459
wit	0.779508	0.294082	-1.688758	-1.749131
fool	0.205584	0.986944	-2.870365	-2.515700

PPMI: Positive PMI

	asyou	twelfth	caesar	henryV
battle	-2.647574	-inf	1.083832	1.331581
soldier	-3.299651	-2.299651	-0.053672	1.622921
good	0.017115	-0.274116	0.356204	0.023459
wit	0.779508	0.294082	-1.688758	-1.749131
fool	0.205584	0.986944	-2.870365	-2.515700

$$\text{PMI}(w, d) = \log \left(\frac{P(w, d)}{P(w)P(d)} \right)$$

$$= \log \left(\frac{\frac{\text{Count}(w, d)}{N}}{\frac{\text{Count}(w)}{N} \times \frac{\text{Size}(d)}{N}} \right)$$

$$= \log \left(\frac{\text{Count}(w, d)}{\frac{\text{Count}(w)}{N} \times \text{Size}(d)} \right)$$

$$\text{PPMI}(w, d) = \max\left(\log \frac{P(w, d)}{P(w)P(d)}, 0\right)$$

PPMI

	asyou	twelfth	caesar	henryV
battle	0.000000	0.000000	1.083832	1.331581
soldier	0.000000	0.000000	0.000000	1.622921
good	0.017115	0.000000	0.356204	0.023459
wit	0.779508	0.294082	0.000000	0.000000
fool	0.205584	0.986944	0.000000	0.000000

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
soldier	1	2	5	25
good	93	76	62	77
wit	21	15	2	3
fool	32	55	2	4

How do these words compare based on cosine?

Raw

	Battle	Soldier	Good	Wit	Fool
Battle	1	0.95	0.66	0.19	0.10
Soldier		1	0.62	0.20	0.15
Good			1	0.85	0.76
Wit				1	0.91
Fool					1

PPMI

	Battle	Soldier	Good	Wit	Fool
Battle	1	0.78	0.68	0	0
Soldier		1	0.065	0	0
Good			1	0.04	0.01
Wit				1	0.54
Fool					1

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
soldier	1	2	5	25
good	93	76	62	77
wit	21	15	2	3
fool	32	55	2	4

How do these documents compare based on cosine?

TF-IDF

	AYLI	Twelfth	Caesar	HenryV
As You	1	0.37	0.97	0.97
Twelfth		1	0.16	0.17
Caesar			1	0.99
HenryV				1

PPMI

	AYLI	Twelfth	Caesar	HenryV
As You	1	0.52	0	0
Twelfth		1	0	0
Caesar			1	0.63
HenryV				1

Sparse Vector Problem

- Words can have similar meanings and appear in “complementary” distributions. Meaning their vectors will be dissimilar
 - The words boat and ship don’t often appear together or even in the same kinds of contexts.
- But many of the words they do co-occur with, do co-occur together in similar contexts
- What we need is a way to reduce the dimensionality of the vectors in a way that reveals these deeper relationships.

Dense Vectors

Why dense vectors?

- Short vectors are easier to use as features in machine learning (fewer weights to learn)
- Dense vectors generalize better than storing explicit counts
- They do better at capturing synonymy among words that appear in different contexts

Variants

- Numerical matrix methods
 - Latent Semantic Analysis/SVD
 - Non-negative matrix factorization
- Probabilistic models
 - Latent Dirichlet Allocation
- Neural learning models
 - Word2vec, GLOVE, BERT, fasttext, etc.

Word2Vec

- One popular embedding method
 - Very fast to train
 - Simple model based on logistic regression

Word2Vec

- Instead of counting how often each word w occurs near "*apricot*"
- Let's **train a classifier** on a binary prediction task:
 - Is w likely to show up in the same kind of context as "*apricot*" ?
 - {apricot, jam}: 1, {apricot, rudder}: 0
- We don't care about this classifier
 - This is called a "pretext" task.
 - We'll use the weights learned by the classifier as our word embeddings

Self-Supervised Training

- Use raw unannotated text as supervised training data!
 - A word w occurring near *apricot* in real text serves as gold ‘correct answer’ to the question
 - “Is word w likely to show up near *apricot* ?”
- No need for hand-labeled annotation
- We’ll see additional examples of self-supervised learning later in the class with LLM training.

Word2Vec: Skip-Gram Task

- Word2Vec provides a variety of options. Here we'll cover:
 - "skip-gram with negative sampling" (SGNS)

Skip-Gram Algorithm

1. Treat a target word and a single neighboring context word as positive examples.
2. Randomly sample other words in the lexicon to get negative samples
3. Use a variant of logistic regression to train a classifier to distinguish those two cases
4. Use the resulting learned feature weights as the embeddings

Skip-Gram Objective

- Given a tuple (t, c) = target word, context word
 - $(\textit{apricot}, \textit{jam})$
 - $(\textit{apricot}, \textit{aardvark})$
- Return the probability that c is a positive context word:
 - $P(+ \mid t, c)$
 - $P(- \mid t, c) = 1 - P(+ \mid t, c)$

How to Compute $p(+ | t, c)$?

■ Intuition

- Capture words as dense vectors.
- Words that are similar in meaning should be near to each other.
- Model similarity with dot-product!
- $\text{Similarity}(t, c) \propto t \cdot c$

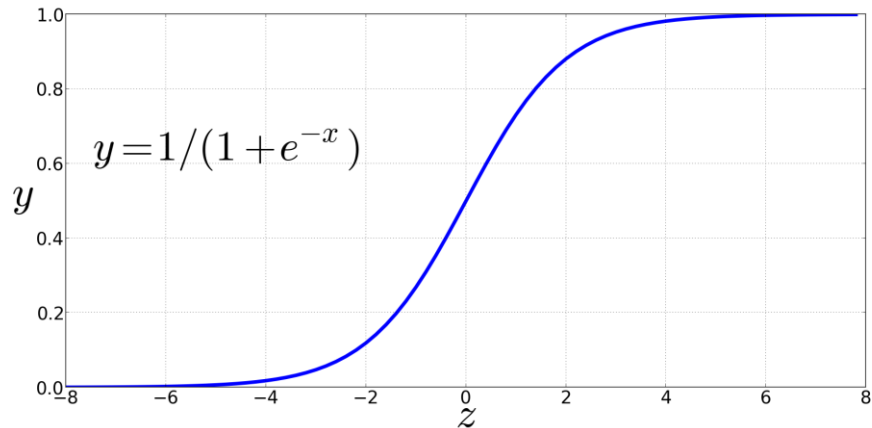
■ Problem

- But the result of a dot product is not a probability, its some arbitrary real number

Dot Product into a Probability

■ Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Skip-Gram Training Data

- Training sentence:
 - ... lemon, a tablespoon of **apricot** jam a pinch ...
 - c1 c2 t c3 c4
- Training data: input/output pairs centering on *apricot*
- Assume a +/- 2 word window

Skip-Gram Training

- Training sentence:
 - ... lemon, a tablespoon of **apricot** jam a pinch ...
 - c1 c2 t c3 c4

positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	preserves
apricot	or

- For each positive example, we'll create k pseudo-negative examples.
- Using *noise* words
- Any random word in the corpus that isn't t , the target word

Skip-Gram Training

- Training sentence:
- ... lemon, a tablespoon of **apricot** jam a pinch ...
- c1 c2 t c3 c4

positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	preserves
apricot	or

negative examples -

k=2

t	c	t	c
apricot	aardvark	apricot	twelve
apricot	puddle	apricot	hello
apricot	where	apricot	dear
apricot	coaxial	apricot	forever

Skip-Gram Training

- Training sentence:

- ... lemon, a tablespoon of **apricot** jam a pinch ...

- c1 c2 t c3 c4

This is a single training instance.

positive examples +

t

c

apricot tablespoon

apricot of

apricot preserves

apricot or

negative examples -

t

c

t

c

apricot aardvark apricot twelve

apricot puddle apricot hello

apricot where apricot dear

apricot coaxial apricot forever

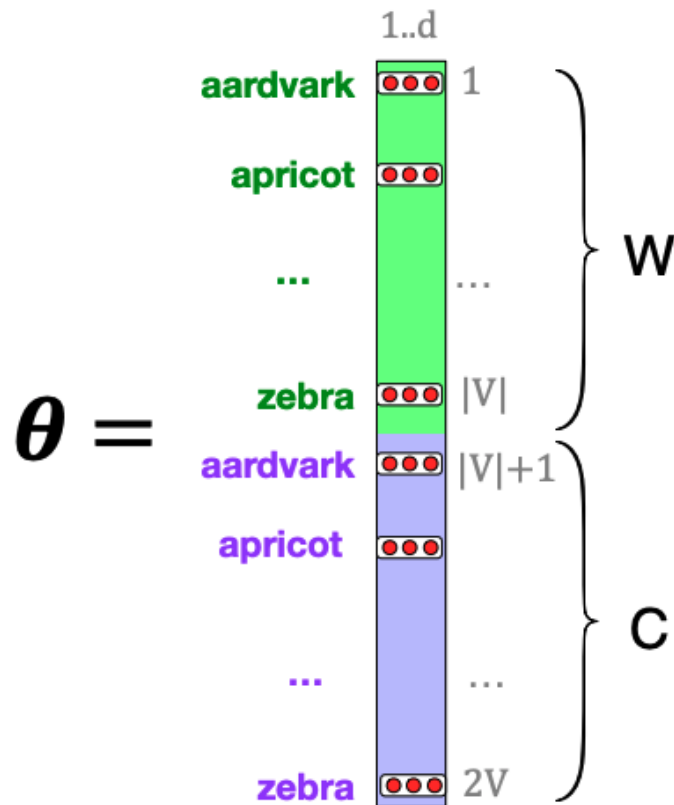
Setup

- Represent words as vectors of some length (say 300), randomly initialized.
 - So, we start with $300 * |V|$ random parameters
 - Really 2 * that. We need both context C and target W vectors
- Our goal to is to learn values for the word vectors that:
 - Maximize the similarity of the **target word**, **context word** pairs (t,c) drawn from the positive data
 - Minimize the similarity of the (t,c) pairs drawn from the negative data.

Learning

- Iterative process. Start with random weights
- Then adjust the word weights to
 - Make the positive pairs more likely
 - More similar
 - And the negative pairs less likely
 - Less similar
- Over the entire training set

Setup

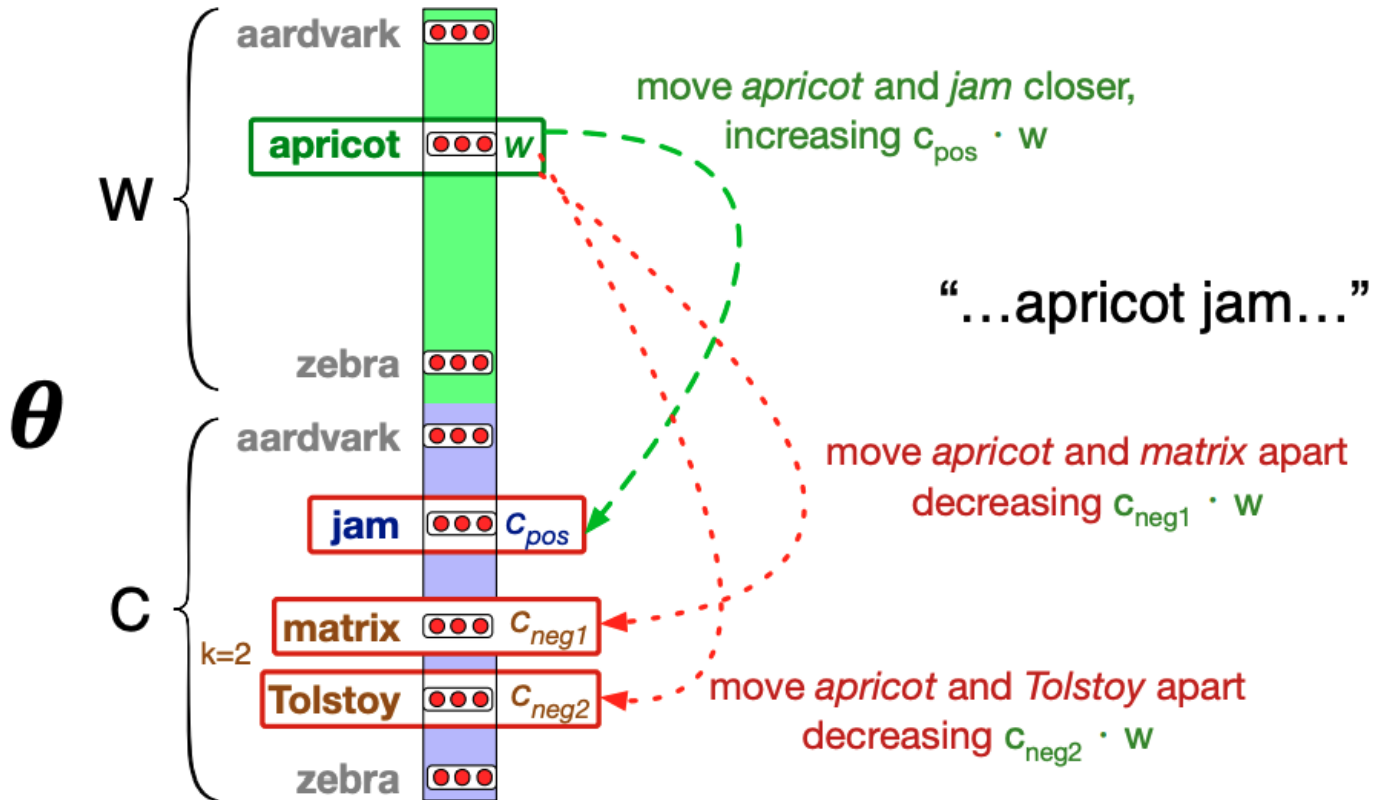


- Each row is a 300-dimensional vector representing a word.
- Separate vectors for each word in V for use as a target word (W) and a context word (C).

Learning

- Iterative process.
- We'll start with random weights
- Then adjust the word weights to
 - make the positive pairs more likely
 - More similar
 - and the negative pairs less likely
 - Less similar
- Over the entire training set

Learning



Objective

- We want to maximize...

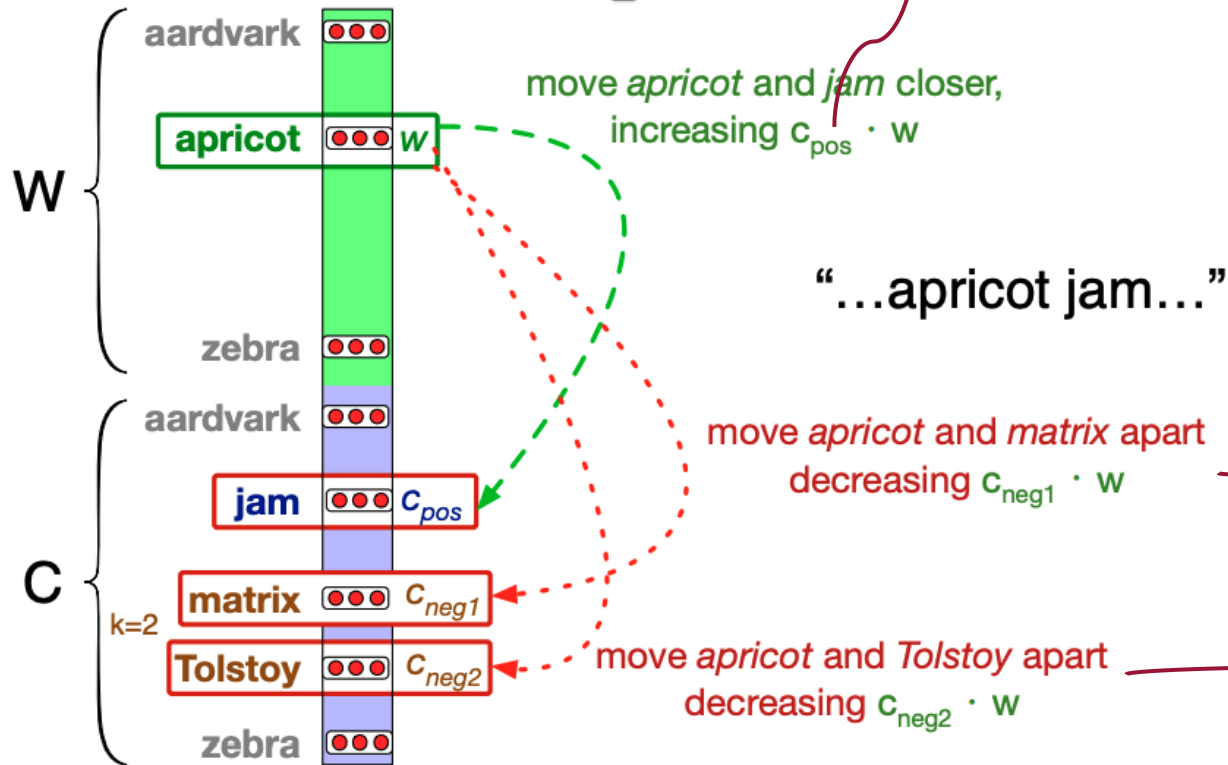
$$\sum_{(t,c) \in +} \log P(+|t, c) + \sum_{(t,c) \in -} \log P(-|t, c)$$

- Maximize the + label for the pairs from the positive training data, and the – label for the pairs sample from the negative data.

LOSS

$$\begin{aligned} L_{CE} &= -\log \left[P(+|w, c_{pos}) \prod_{i=1}^k P(-|w, c_{neg_i}) \right] \\ &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log P(-|w, c_{neg_i}) \right] \\ &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log (1 - P(+|w, c_{neg_i})) \right] \\ &= - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right] \end{aligned}$$

$$-\left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$



Gradient

- Recall from logistic regression... for the current weights and a training example x
 - - (correct – computed) x
 - That's the gradient used for the weight update
- But here we're taking the dot product of the word vector w and the context vector c
 - So, what plays the role of x ?

Partial Derivatives

- The point of the partial derivative is to measure how much the value of a function changes given some small change in a single variable. Holding the other variables fixed.
- That's why we talk about the partial derivative with respect to a particular variable. So a function F with three inputs x , y and z gives

$$\frac{\partial F}{\partial x}, \frac{\partial F}{\partial y}, \frac{\partial F}{\partial z}$$

Gradient

- For learning we need the partial derivative of the loss with respect to the weights, both $\mathbf{c}$ and $\mathbf{w}$...

$$\frac{\partial L_{CE}}{\partial \mathbf{c}_{pos}} = [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1] \mathbf{w} \quad (6.35)$$

$$\frac{\partial L_{CE}}{\partial \mathbf{c}_{neg}} = [\sigma(\mathbf{c}_{neg} \cdot \mathbf{w})] \mathbf{w} \quad (6.36)$$

$$\frac{\partial L_{CE}}{\partial \mathbf{w}} = [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1] \mathbf{c}_{pos} + \sum_{i=1}^k [\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w})] \mathbf{c}_{neg_i} \quad (6.37)$$

Weight Update

- Given those gradients we update C and W

$$\mathbf{c}_{pos}^{t+1} = \mathbf{c}_{pos}^t - \eta [\sigma(\mathbf{c}_{pos}^t \cdot \mathbf{w}^t) - 1] \mathbf{w}^t \quad (6.38)$$

$$\mathbf{c}_{neg}^{t+1} = \mathbf{c}_{neg}^t - \eta [\sigma(\mathbf{c}_{neg}^t \cdot \mathbf{w}^t)] \mathbf{w}^t \quad (6.39)$$

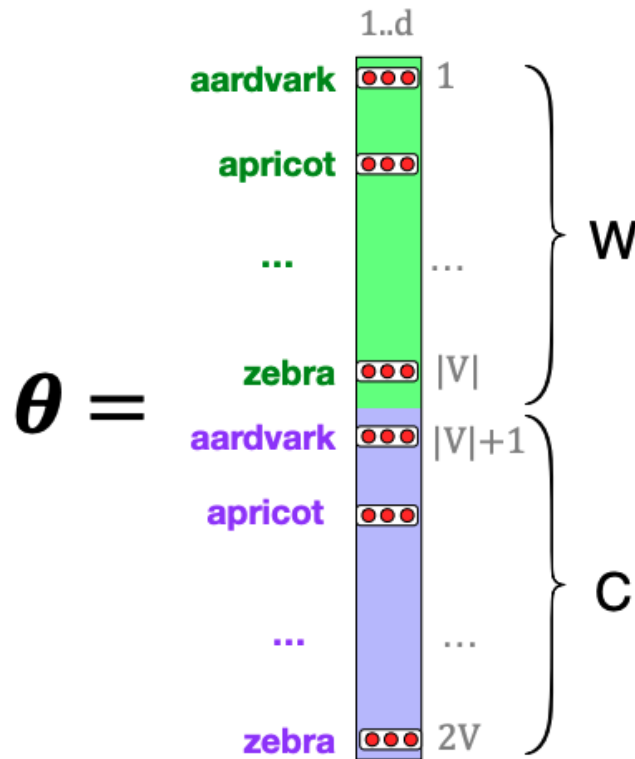
$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \left[[\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}^t) - 1] \mathbf{c}_{pos} + \sum_{i=1}^k [\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w}^t)] \mathbf{c}_{neg_i} \right] \quad (6.40)$$

Skip-Gram Algorithm

1. Treat a target word and a neighboring context word as positive examples.
2. Randomly sample other words in the lexicon to get negative samples
3. Train a classifier to distinguish those two cases using CE Loss
4. Use the resulting feature weights as the embeddings.

Which Set of Weights?

- The result of the training is two sets of embeddings, one for each word as a target and one for each word as a context word.
 - Throw away C and use W
 - Combine them somehow, add or average...



Summary

■ **Concepts (Word senses)**

- Have a complex many-to-many association with **words** (homonymy, multiple senses)
- Have many relations with each other
 - *Synonymy, Antonymy, Superordinate*
- But are hard to define formally (necessary & sufficient conditions)

■ **Embeddings = vector models of meaning**

- 1 static vector for each entry in the vocabulary
- More fine-grained than just a string or index
- Especially good at modeling similarity/analogy
 - *Just download them and use cosines!!*
- Can use sparse models (tf-idf) or dense models (word2vec, GLoVE)